Final Project Report Template

1. Introduction
    - 1.1. Project overviews
    - 1.2. Objectives
2. Project Initialization and Planning Phase
    - 2.1. Define Problem Statement
    - 2.2. Project Proposal (Proposed Solution)
    - 2.3. Initial Project Planning
3. Data Collection and Preprocessing Phase
    - 3.1. Data Collection Plan and Raw Data Sources Identified
    - 3.2. Data Quality Report
    - 3.3. Data Preprocessing
4. Model Development Phase
    - 4.1. Model Selection Report
    - 4.2. Initial Model Training Code, Model Validation and Evaluation Report
5. Model Optimization and Tuning Phase
    - 5.1. Tuning Documentation
    - 5.2. Final Model Selection Justification
6. Results
    - 6.1. Output Screenshots
7. Advantages & Disadvantages
8. Conclusion
9. Future Scope
10. Appendix
    - 10.1. Source Code
    - 10.2. GitHub & Project Demo Link

Natural Language Interfaces for Databases

---

1. Introduction

1.1 Project Overview:

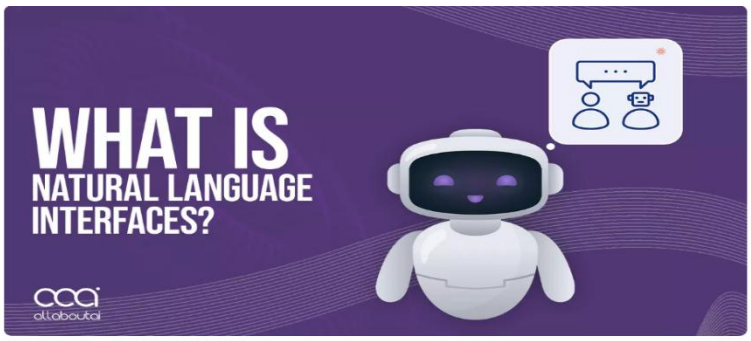
Natural Language Interfaces for Databases (NLIDB) is a system that allows users to interact with databases using natural language instead of complex query languages like SQL or MongoDB queries. The system converts human language questions into database queries and retrieves the corresponding results.

This project aims to make database interaction simpler, faster, and more accessible for users who do not have knowledge of database query languages. By using Natural Language Processing (NLP) techniques, the system understands user input such as:

- “Show all students”
- “Find students with marks greater than 80”
- “Display students from CSE department”

The system then automatically converts these inputs into MongoDB queries and retrieves the appropriate data from the database.

A user-friendly web interface built using Streamlit allows users to interact with the system easily.



1.2 Objectives :

The main objectives of the project are:

1. Simplify Database Interaction
   Allow users to query databases using natural language instead of writing complex database queries.
2. Improve Accessibility
   Help non-technical users interact with databases without prior knowledge of MongoDB or SQL.
3. Efficient Query Processing
   Convert natural language queries into valid MongoDB queries automatically.
4. Interactive User Interface
   Provide an easy-to-use web interface where users can ask questions and view results instantly.
5. Real-Time Data Retrieval
   Fetch and display database results quickly based on user input.

2. Project Initialization and Planning Phase

2.1 Define Problem Statement :

Modern databases store huge amounts of data, but accessing this information often requires knowledge of database query languages such as SQL or MongoDB queries. Many users, especially non-technical users, find it difficult to write these queries.

This creates a gap between users and database systems, making it challenging to retrieve information quickly.

The problem is to develop a system that allows users to interact with databases using natural language queries, eliminating the need to learn complex database syntax.

Key Problems Addressed:

1. Difficulty in writing database queries.
2. Lack of technical knowledge among users.
3. Time-consuming manual query writing.

2.2 Project Proposal (Proposed Solution) :

The proposed solution is to build a Natural Language Interface for MongoDB Database that translates user questions into MongoDB queries.

The system will include:

1. Natural Language Processing

The system will process user input and extract keywords and intent.

2. Query Generation

User questions will be converted into MongoDB queries automatically.

Example:

User Input:

Show students with marks greater than 80

Generated MongoDB Query:

db.students.find({"marks": {"$gt": 80}})

3. Database Integration

MongoDB will store the dataset (student information).

4. User Interface

A Streamlit web application will allow users to enter queries and view results.

2.3 Initial Project Planning :

Project Milestones

1. Requirement Analysis

- Identify system requirements
- Define project scope

Deliverable: Requirement document

---

2. System Design

- Design system architecture
- Plan database schema
- Design UI

Deliverable: Design documentation

---

3. Development Phase

- Implement NLP logic
- Connect MongoDB database
- Develop Streamlit interface

Deliverable: Working prototype

---

4. Testing Phase

- Test query accuracy
- Verify database connectivity
- Fix errors

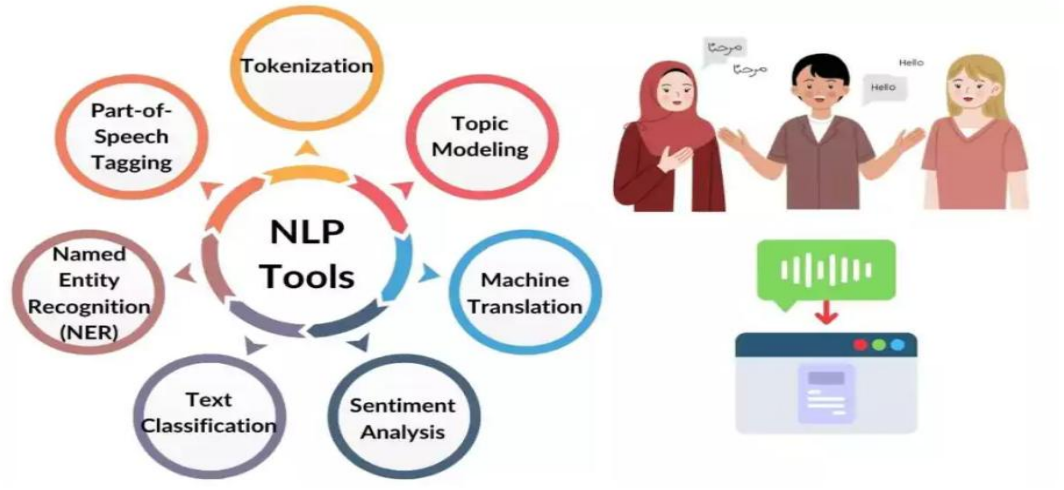
Deliverable: Tested system

---

5. Deployment

- Run the application locally
- Demonstrate project functionality

Deliverable: Fully working application



3. Data Collection and Preprocessing Phase

3.1 Data Collection Plan and Raw Data Sources :

For this project, a student dataset is used.

Data Fields

The dataset includes:

- Student ID
- Name
- Department
- Marks
- Age

Data Source

The data is stored in a MongoDB database collection named "students".

Example document:

```
{
  "name": "Rahul",
  "department": "CSE",
  "marks": 85,
  "age": 21
}
```

Data Collection Method

Data is manually inserted into MongoDB using:

- MongoDB Compass
- MongoDB Shell
- Python scripts

3.2 Data Quality Report :

The dataset was evaluated using the following criteria:

Completeness

All student records contain required fields such as name, department, marks, and age.

Consistency

Data formats were standardized.

Example:

Department values such as:

CSE

ECE

EEE

Accuracy

Data values were verified before insertion.

Duplication

Duplicate records were removed.

3.3 Data Preprocessing :

The preprocessing steps include:

1. Cleaning the dataset
2. Removing duplicate entries
3. Standardizing text values
4. Formatting numeric fields

---

4. Model Development Phase

4.1 Model Selection Report :

The project uses Natural Language Processing techniques instead of traditional machine learning models.

The system processes user input using rule-based logic to identify keywords such as:

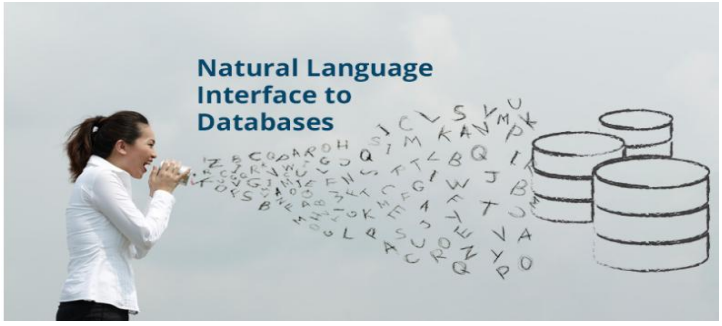
- show
- find
- greater than
- less than
- department

These keywords are then mapped to MongoDB query operators.

Example mapping:

Natural Language	MongoDB Operator
greater than	$gt
less than	$lt
equal to	$eq

This approach is lightweight and suitable for small database systems.



4.2 Initial Model Training Code, Validation and Evaluation

The system processes user queries and converts them into MongoDB queries.

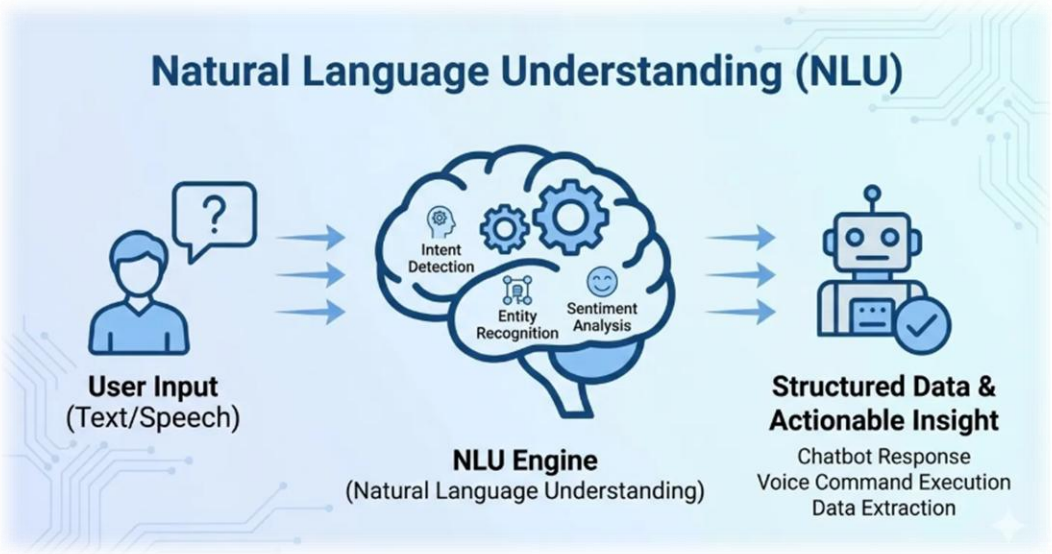
Example code snippet:

```
def generate_query(user_input):
    if "greater than" in user_input:
        return {"marks": {"$gt": value}}
```

The generated query is then executed on MongoDB using Python.

Libraries Used

- Python
- Streamlit
- PyMongo
- python-dotenv



5. Model Optimization and Tuning Phase

5.1 Tuning Documentation :

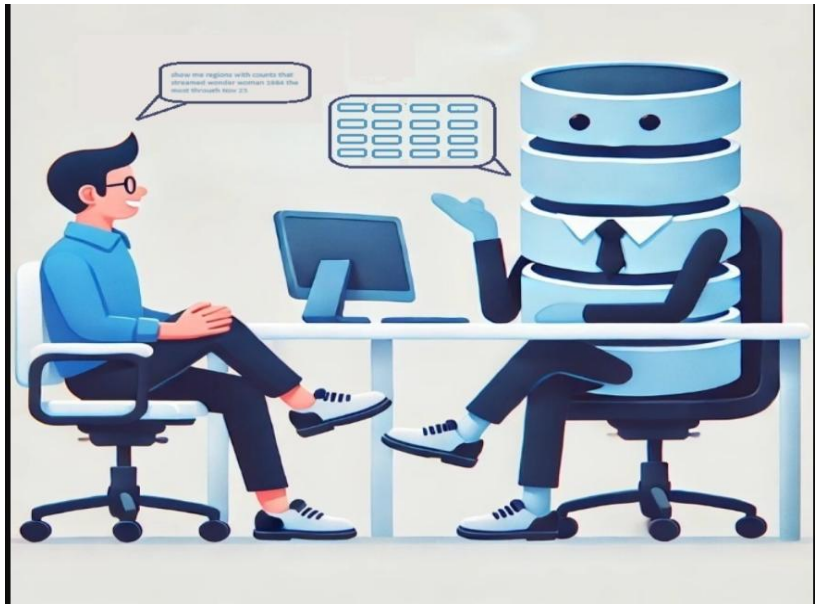
Optimization techniques used:

1. Improved keyword detection
2. Added more query patterns
3. Handled multiple user inputs
4. Reduced processing time

5.2 Final Model Selection Justification :

The rule-based NLP approach was selected because:

- It is simple and lightweight
- Easy to implement
- Requires minimal training data
- Works efficiently for structured queries



6. Results :

The system successfully converts natural language queries into MongoDB queries and retrieves data.

Example Query:

User Input:

Show students from CSE department

Output:

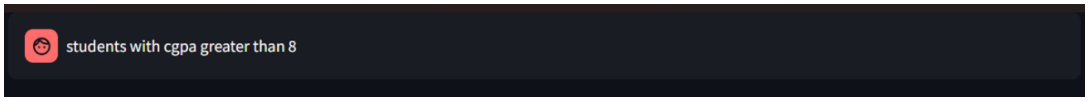
List of students belonging to the CSE department.

6.1 Output Screenshots :

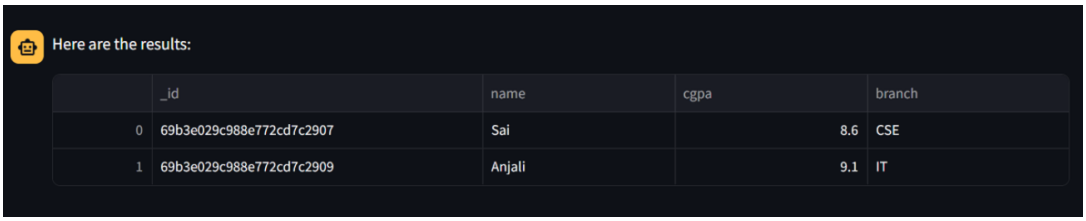
Include screenshots of:

1. Application Home Page:

2. User Query Input:



3. Generated Query:



	_id	name	cgpa	branch
0	69b3e029c988e772cd7c2907	Sai	8.6	CSE
1	69b3e029c988e772cd7c2909	Anjali	9.1	IT

4. Database Results:

	_id	name	cgpa	branch
0	69b3e029c988e772cd7c2907	Sai	8.6	CSE
1	69b3e029c988e772cd7c2908	Ravi	7.2	ECE
2	69b3e029c988e772cd7c2909	Anjali	9.1	IT
3	69b3e029c988e772cd7c290a	Kiran	8	CSE

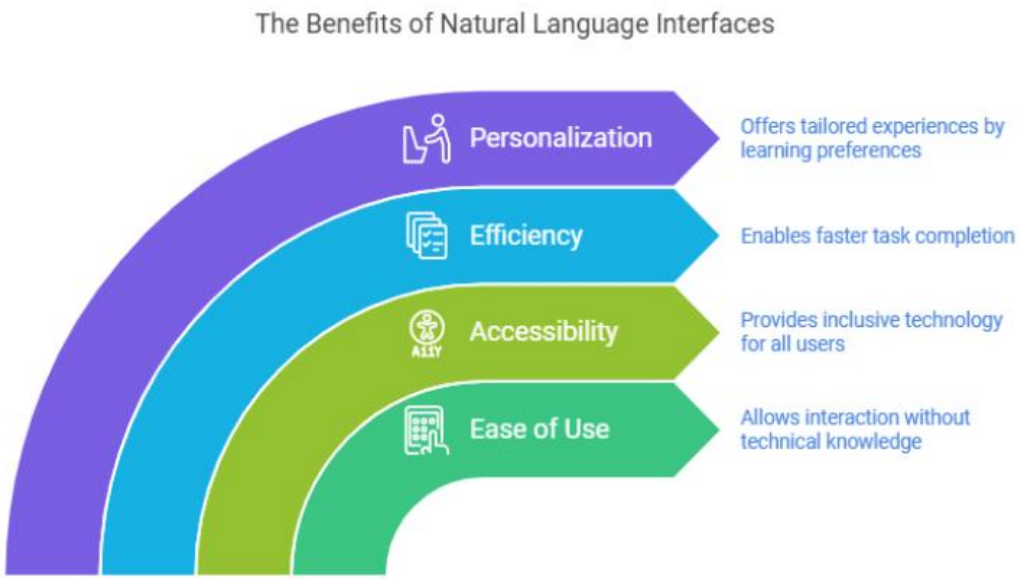
7. Advantages & Disadvantages :

Advantages :

1. Easy database interaction
2. No need to learn database query languages
3. Faster data retrieval
4. User-friendly interface
5. Accessible for non-technical users

Disadvantages :

1. Limited query complexity
2. Requires predefined query patterns
3. May not understand very complex sentences
4. Limited dataset support



8. Conclusion :

The Natural Language Interface for Databases project demonstrates how natural language processing can simplify database interactions.

By converting user questions into MongoDB queries, the system allows users to retrieve data easily without writing complex queries.

The project successfully integrates Python, MongoDB, and Streamlit to provide a user-friendly interface for database querying.

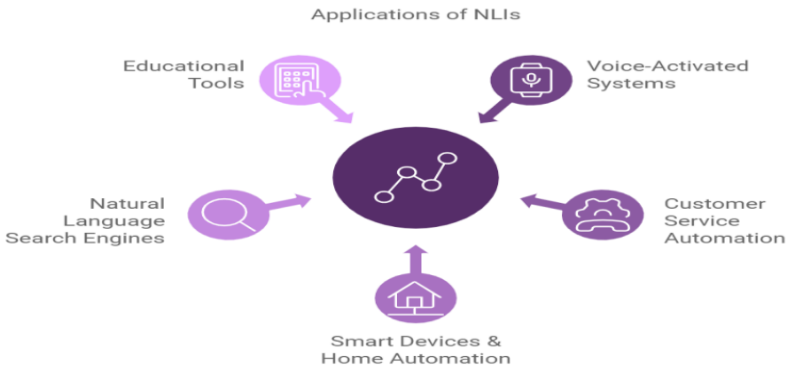
This approach improves accessibility and efficiency for users who lack technical knowledge of database systems.

---

9. Future Scope :

Future improvements include:

1. Integration with large language models
2. Support for multiple databases
3. More complex query understanding
4. Voice-based database querying
5. Deployment as a web application



10. Appendix :

10.1 Source Code :

The source code includes:

- app.py

nl2mongo > app.py

```python
import streamlit as st
from main import process_user_input, llm_handler
import pandas as pd

st.set_page_config(page_title="Compliance Regulatory MongoDB Query Assistant", layout="wide")
st.title("MongoDB Chat Assistant")
# Initialize session state
if "messages" not in st.session_state:
    st.session_state.messages = []
if "chat_input" not in st.session_state:
    st.session_state.chat_input = ""


# Function to reset chat
def reset_chat():
    st.session_state.messages = []
    st.session_state.chat_input = ""


# Sidebar for options
st.sidebar.title("Options")
if st.sidebar.button("New Chat"):
    reset_chat()
    llm_handler.conversation_history = []  # Clear LLM context when starting a new chat

# Display chat messages
for msg in st.session_state.messages:
    role = "You" if msg["role"] == "user" else "Assistant"
    with st.chat_message(role):
        st.markdown(msg["content"])
        if msg.get("results"):  # Display query results
            df = pd.DataFrame(msg["results"])
            if "_id" in df.columns:
                df["_id"] = df["_id"].astype(str)  # Convert ObjectId to string
            st.dataframe(df, use_container_width=True)  # Render as a table

# Add the chat input field
```

- main.py

```python
import os
import logging
from utils.llm_handler import LLMHandler
from utils.mongodb_handler import MongoDBHandler

LOG_FILE = os.getenv('LOG_FILE')

# Configure logging
logging.basicConfig(
    filename=LOG_FILE,
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s]: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

# Initialize LLMHandler
llm_handler = LLMHandler(
    model_name="gpt-4o-mini",
    temperature=0,
)

# Initialize MongoDBHandler
mongodb_handler = MongoDBHandler(
    database_name=os.getenv("DATABASE_NAME"),
    collection_name=os.getenv("COLLECTION_NAME"),
)


def process_user_input(user_input):
    """
    Process user input, generate a query via LLM, execute it on MongoDB, and return results.

    :param user_input: The user's natural language input.
    """

    response = llm_handler.chat_with_llm(user_input)
```

- llm_handler.py

nl2mongo > utils > llm_handler.py > LLMHandler > chat_with_llm

```python
import logging


class LLMHandler:
    def __init__(self, *args, **kwargs):
        """
        Initialize a simple rule-based handler instead of OpenAI.
        """
        logging.info("Rule-based query handler initialized.")
        self.conversation_history = []

    def chat_with_llm(self, user_input: str) -> dict:
        """
        Convert natural language input into a MongoDB query.
        """

        try:
            user_input_lower = user_input.lower()

            # Query 1: Show all students
            if "all students" in user_input_lower or "show students" in user_input_lower:
                return {
                    "query": {},
                    "explanation": "Showing all students"
                }

            # Query 2: CGPA greater than
            if "cgpa greater than" in user_input_lower:
                return {
                    "query": {"cgpa": {"$gt": 8}},
                    "explanation": "Finding students with CGPA greater than 8"
                }

            # Query 3: CSE department
            if "cse" in user_input_lower:
                return {
                    "query": {"department": "CSE"},
```

- database_handler.py

nl2mongo > utils > mongodb_handler.py > ...

```python
import os
from pymongo import MongoClient
from typing import Dict, Any, List
import logging


class MongoDBHandler:
    def __init__(self, database_name: str, collection_name: str):
        """
        Initialize the MongoDB connection.

        :param database_name: The name of the database to connect to.
        :param collection_name: The name of the collection to operate on.
        """
        try:
            # Connect to MongoDB
            self.client = MongoClient("mongodb://localhost:27017")

            # Select database
            self.database = self.client["studentdb"]

            # Select collection
            self.collection = self.database["students"]

            logging.info("MongoDB connection established successfully.")

        except Exception as e:
            logging.error(f"Error initializing MongoDB connection: {e}")
            raise

    def execute_query(self, query: Dict[str, Any], sort: Dict[str, int] = None) -> List[Dict[str, Any]]:
        """
        Execute a MongoDB query with optional sorting and return the results.
        """
        try:
            logging.info(f"Executing query: {query}, with sort: {sort}")
```

10.2 GitHub & Project Demo Link:

(https://github.com/jsaiteja4074/Natural-Language-Interfaces-for-Databases-Hack-N-Go-with-MongoDB/tree/main)

https://drive.google.com/file/d/1zMcrTQ8UZZc0bLA4SvkXdhb0PRzGJWig/view?usp=drive_link